Date: Day M T W T F S

Past-Papers 2023

1) Define front end. Solved
2) Differentiate between passe tree (Solved) and abstract syntax tree.
3) Give some uses of symbol tables. (Solved)
4) Why hash tables are implemented? (Solved)
5) Write the regular expression for floating point numbers. (Solved)
6) In which phase of compiles, a CFG is used? (Solved)
7) Define interprocedural optimization. (Solved)
8) What is implicit deallocation? (Solved)
9) What is output of lexical analyzers? (Solved)
10) Give types of list sheduling Solved)
11) Write the regular expression (Solved) for identifiers that are strings of letters, underscores or digits

beginning with non-digit?

12) What is peephole optimization?
(Solved)

13) Give three problems solved by the Compilers back end to generate executable code?
(1) Code optimization
(2) Code Generation
(3) Register Allocation

14) Define vectors?
- Vectors refers to a dynamic array or a resizable array data structure.
- Used to represent sequences of elements that can dynamically grow or shrink in size during the compilation process.
- Vectors are commonly employed to store various kinds of information such as symbol tables, ICR, and other data structures.
- Provide a flexible and efficient way to manage collection of data, allowing the compiler to adapt the changing requirements as it processes the source code.
- Facilitates the efficient organization and manipulation of data within the compilers internal working.

15) Write the three address code of:

$A = d - c * s$

$t_1 = c * s$

$A = d - t_1$ // t_1 is temporary variable

OR

$t_1 = c * s$

$t_2 = d - t_1$

$A = t_2$

Date: Day: M T W T F S

16) Complete the following statement
z* _____ = (z<<4) + z

⇒ The statement is a bitwise left shift operation.

⇒ The complete statement would be:

z*17 = (z<<4) + z

⇒ This expression is equivalent to multiplying z by 17 using a combination of bitwise left shift (<<) and addition.

Subjective Part

2. How scanning and passing combined used for:

```
main(){
int a,b;
a=b;
}
```

Solved already.

Date: Day: M T W T F S

1. Explain the structure of compiler?

Solved already.

2. Write three address code for the following:

```
for( i=1 ; i<=10;i++)
    x=y+z
```

(1) i=1
(2) L1 : if i<=10 goto L2
(3) x = y+z
(4) i = i+1
(5) goto L1
(6) L2 : exit

OR

(1) i=1 ✓
(2) L1: t1 = y+z
(3) x = t1
(4) t2 = i+1
(5) i = t2
(6) if i<=10 goto L1
(7) goto --

Date: Day: M T W T F S

Q. Optimize the following code:

```
v = P^3
b = 4
x = P
d = x*x*x
e = b*16
f = v+d
g = e*f
```

Ans: After removing exponential power & copy propagation.

```
v = P*P*P
b = 4
d = P*P*P
e = b*16
f = v+d
g = e*f
```

After constant propagation.

```
v = P*P*P
b = 4
d = P*P*P
e = 64   // 4*16 = 64
f = v+d
g = e*f
```

Date: Day: M T W T F S

After Common Sub expression elimination

```
v = P*P*P
b = 4
d = v
e = 64
f = v+d
g = e*f
```

After
- Dead code elimination
- Constant folding

Optimized code
Constant folding

```
v = P*P*P
f = d+v
g = 64*f
```

2. Difference between multi processing and rational preprocessor?

Date: Day: M T W T F S

Past-Papers 2019 to 17 from ch (code optimization, IMCG, etc ...).

2. What is the function of static checker?

⇒ Static checker is a component of a compiler that analyzes the source code without executing it.

⇒ Its primary function is to perform static analysis, examining the code for various types of issues or violation before the program is run.

⇒ The static checker operates on the source code before the actual compilation process begins to catch and fix potential issues early.

⇒ Some functions of static checker are as follows:

(1) Syntax checking
(2) Semantic analysis
(3) Type checking
(4) Scope checking
(5) Code flow checking
(6) Dead code elimination
(7) Constant folding

2. What is dead code elimination?

Solved

2. Why we use symbol table in compiler?

Solved

2. Mention some important properties of IMC?

Solved.

2. What are the benefits of generating intermediate code?

Solved.

2. What is difference b/w machine independent and machine dependent code?

Date: Day: M T W T F S

Machine IC	Machine DC
Target Platform:	
○ Designed to be portable across different hardware.	○ Specifically tailored for a particular target machine or architecture.
Abstraction level:	
○ Typically higher-level and abstract, focusing on algorithms and data structures.	○ Lower level, dealing with machine specific instructions and optimization.
Portability:	
○ High portability, code can be compiled and executed on various platforms without modification.	○ Tied to specific platform, may require modification for different machines.
Development flexibility:	
○ More	○ less

Date:

Optimizations:	
○ Generally less optimized for a specific machine but can benefit from general compiler optimization.	○ Optimized for a particular machine, taking advantages of its architecture and instruction set.
Examples	
○ High level programming languages like C, Java, Python.	Assembly language, ○ machine code. ○ Code written in low level language.

1. Write the three-address code of A% = 10

⇒ Given code:

A% = 10

⇒ Let t_1 is temporary variable

$t_1 = A \% 10$

$A = t_1$

Q. What is kind of analysis is performed by static checkes?

- Syntax Analysis
- Semantic Analysis
- Type checking
- Code style checking
- Constant folding
- Dead code elimination.

Q. Whath types of rules are included in automatic code generator?

⇒ Automatic code generators use predefined rules or templates to convert high-level specifications or models into executable code.

⇒ The types of rules included in an automatic code generator depends on the specific requirements and characteristics of the target language, platform or domain.

Some common types of rules found in automatic code generator are:

- mapping rules
- Code generation Templates
- Data mapping rules
- Dependency rules
- Naming Conventions
- Error handling Rules
- Optimization rules
- Platform - specific rules
- code formatting rules
- Documentation rules
- Testing rules

Q. What is the role of Symbol table manager in Compiler?

⇒ Primary role of symbol table manager is to manage:

- Symbol storage
- Scope management
- Duplicate detection

- Attribute Retrieval
- Type checking
- Memory Allocation
- Error reporting
- Optimization Support

Long

Q. Write note on error recovery techniques?

Solved

Q. Write detail note on code Optimization?

Solved

Q. Write a three address code for the following statement:

```
int addition (int a, int b);
int main()
{
    int num1, num2, add;
    cout<<"Enter two numbers";
    cin>>num1>>num2;
    Add= addition (num1,num2);
    cout<<" Addition is "<<add<<endl;
    return 0;
}
int addition (int a, int b)
{
    return a+b;
}
```

Ans:

(1) call main
(2) Param num1
(3) Param num2
(4) Add= call addition, 2
(5) t1= a+b
(6) RETURN t1

```
cout<<"Enter two numbers"
cin>>num1
cin>>num2
Param num1
Param num2
t1= call addition, 2
assign add, t1
t1= a+b
return a+b
cout<<"Addition is"
cout<<add
cout<<endl
return 0
```

Q. Optimize the following code:

```
int j, x, y, z;
x = 0;
y = 1;
z = 1;
for (j=0 ; j<=N ; j++)
{
    x = (j+1) * (j+1) + (j+1) * x + (8*a/b) * j;
    x = x + z * y;
}
```

Code movement:

- The statement x = x + z * y can be do in the following way

```
temp2 = z * y
x = x + temp2
```

And put temp2 outside the loop.

Loop invariant:

⇒ The expression 8*a/b can be avoided overhead place it outside the loop.

```
temp1 = 8 * a/b
```

Common sub expression elimination:

⇒ The code (j+1) * (j+1) * (j+1) can be replaced by:

```
3 * (j+1)
```

to reduce the calculation.

Optimize code is:

```
int j, x, y, z;
x = 0;
y = 1
z = 1
float temp1 = 8 * a/b;
int temp2 = z * y
for ( j=0 ; j<=N ; j++)
{
x = 3*(j+1)*x + temp1 * j;
x = x + temp2; }
```

Date: Day: M T W T F S

Q Optimize the following Code with title of the technique you use:

```
x = 1
y = a*b+3
z = a*b+x+z+2
x = 3
```

Constant folding:

In statement:

$z = a*b + x + z + 2$

we replace x by x = 1

$z = a*b + 1 + z + 2$

$z = a*b + z + 3$

$z = a*b + 3 + z$

Common Sub expression elimination:

In statement:

$z = a*b + 3 + z$

And

$y = a*b + 3 + z$

Date: Day: M T W T F S

have same sub expression so we can remove

$y = a*b+3$ OR $z = a*b+3$ Remain

$z = y + z$

Code optimize is:

$x = 1$

$y = a*b+3$ OR

$z = y + z$

$x = 3$

```
x = 1
y = a*b+3
z = a*b+2+y
x = 3
```

Dead code eliminations

After dead code elimination:

$y = a*b+3$

$z = y + z$

OR

$z = a*b+3+z$

Date: Day: M T W T F S

Given the following CFG:

Exp → Exp + Term

Exp → Exp − Term

Exp → Term

Exp → Exp * Factor

Exp → Exp / Factor

Exp → Factor

Exp → (Exp)

Exp → id

Write an attribute grammar that calculates the value of any arithmetic expression with the following restriction:

Arithmetic Operation can only be allowed when both operands of the operator have same data type. Otherwise an error should be generated.

Exp.val = Exp.val + Term.val

Exp.val = Exp.val − Term.val

Exp.val = Term.val

Exp.val = Exp.val * Factor.val

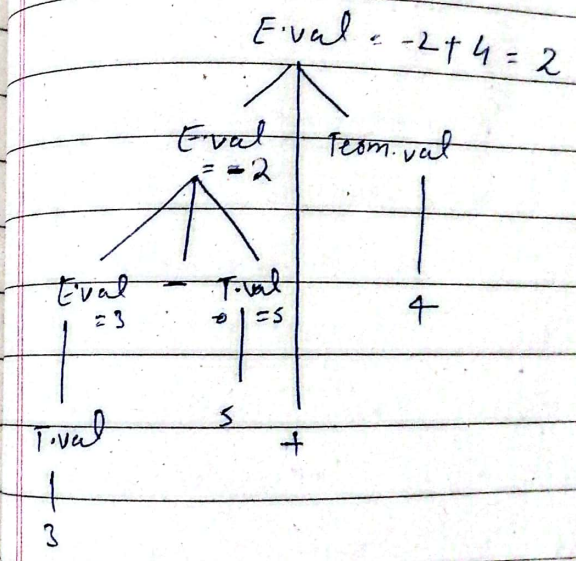
Exp.val = Exp.val / Factor.val

Exp.val = Factor.val

Exp.val = Exp.val

Exp.val = id.len

Let w = 3 − 5 + 4



Date: PP-2020 Day: M T W T F S

(i) Explain recursive Grammar, with help of an Example?

⇒ Recursive grammar refers to a type of grammar where a production rule in the grammar allows the definition of a symbol in terms of itself.

In other words:

⇒ A grammar is recursive if it permits the derivation of strings that include the same non-terminal symbol on both sides of a production.

e.x:

Exp → Exp + Term | Term

eg:,

Exp → Term + Exp | Term

Term → Factor * Term | Factor

Factor → (Exp) | Number

ex: S → SaS, S → ε

If the CFG can generate infinite number of strings then the grammar is called recursive grammar

Write Steps to create a TD for a predictive Parser?

- Grammar Analysis
- First and Follow sets
- Construct the passing table
- identify states
- Build transition diagram
- Start & Accept States
- Handle ambiguities
- Validate and Test

Describe the ambigious grammar?

Solved

Give any use of TD?

Solved

How java Compiler works?

The javac translates human readable java source code into platform independent bytecode. This bytecode is then executed by the JVM, allowing java programs to run on different Platforms without modification.

Q. Define Cross compiler?

Solved

Q. Differentiate between Macro-Processor and Rational Pre-Processor?

Solved

Q Differentiate the Recursive Decent Parser and Predictive Parser?

Solved

Q. Difference between linker and loader?

Solved

Q. Difference between Terminal and non-Terminals?

Solved

Q Name phases of Compiler?

Solved

Q Describe attribute grammar framework?

Solved

Q How tokens are validated?

⇒ The process of token validation is typically carried out by the lexical analyzer or lexer, which is the initial phase of a compiler.

⇒ Tokens are validated by defining rules based on the language syntax.

⇒ If any deviations or errors are reported, and the compiler can take appropriate actions to handle these situations.

Q How many was ways passers can be implemented ? Name these implementations?

⇒ Passers can be implemented in the following ways:

Passers

Top-Down

Bottom up

Date: Day: M T W T F S

Q. What is scope of code Optimization?

Solved already

Q. Differentiate Local Register Allocation and Global Register allocation?

Local RA	Global RA
Scope of analysis:	
o Limited to a single basic block or function	o Considers the entire program or large sections of it.
Optimization level:	
o local optimizations within a function	o Global optimizations across the entire program
Compiler Phase:	
o Often part of the middle end or backend	o Usually Part of the backend or later optimization Phases.
Register Spilling:	
o Less likely to involve register spilling	o may need to handle register spilling more extensively.
Example algorithms:	
o Local graph coloring.	o Chaitin Briggs, George etc.

Long

Q Build the parse tree for the expression 5+(2+3) using the following grammar

E → int | (E) | E+E

Given expression: 5+(2+3)